FINAL YEAR PROJECT · MID-1 REVIEW

Autonomous Cybersecurity Analyst AI

An AI-driven static analysis and LLM reasoning pipeline for automated vulnerability detection, explanation, and patch generation.

Krunal Parmar
23CP044

Sumit Trapasiya
23CP070

Rushi Divecha
23CP201

Panel :
Mohsin I Hasan
Dr. Mahasweta J Joshi
Zalak S Parmar

Course 4CP31 · Mid-1 Presentation · August 2026

Introduction to the Project

Why manual vulnerability triage doesn't scale — and what we're building instead

PROBLEM

Codebases grow faster than security teams can review them.

- → Manual code review for security flaws is slow and expertise-heavy
- → Traditional static analyzers flag issues but rarely explain impact
- → Developers get a rule ID, not a fix — remediation is left to them
- → Small teams / student & OSS projects can't afford enterprise AppSec tools

Introduction to the Project

Why manual vulnerability triage doesn't scale — and what we're building instead

OUR APPROACH

Close the loop: detect → reason → explain → patch.

- **Static Detection**
- **LLM Reasoning**
- **Patch Generation**
- **Automated Reporting**

Literature Survey / Study of Existing Systems

Positioning against established static-analysis and AI-assisted security tools

System	Approach	Explains findings?	Suggests patches?	Cost
Bandit / Semgrep	Pure static rule matching (AST/regex)	No — rule ID only	No	Free / OSS
SonarQube	Static analysis + quality gates	Partial — generic advice	Limited	Free tier + paid
Snyk Code	Static + proprietary ML ranking	Yes, templated	Yes, limited	Paid / freemium
GitHub Copilot Autofix	LLM-assisted suggestions in CI	Yes	Yes	Paid (GH Advanced Security)
Our System	**AST detection + LLM reasoning pipeline**	**Yes — contextual, per finding**	**Yes — schema-validated**	**Free-tier API, no card**

Gap identified: Existing free/OSS tools detect but don't explain or fix. Tools that explain and fix are paid, enterprise-gated, or need cloud accounts. Our pipeline targets the same detect → explain → patch capability using a free-tier LLM API, making it viable for students, small teams, and OSS maintainers.

Tentative Modules of the Project

M1–M4 implemented and verified this phase · M6–M7 planned for Phase 4+

Module - 1

Ingestion

Repository/file ingestion, language detection, source normalization for the analysis pipeline

Module - 2

Detection Layer

AST + tree-sitter rule engine — SQLI, SECRET, XSS, AUTH, DESER/SSRF classes

Tentative Modules of the Project

Built Planned

M1–M4 implemented and verified this phase · M6–M7 planned for Phase 4+

Module -3

Reasoning & Patch Engine

Groq LLM reasoning over findings; structured, schema-validated patch suggestions

Module - 4

Reporting

Jinja2-templated JSON + Markdown report generation per scan

Module - 5

Orchestrator / CLI

Bounded linear pipeline control — wires M1→M2→M3→M4, CLI entry point

Implementation

Phase 1–3 minimal pipeline — built and verified against demo_app

BUILT & VERIFIED

- **All 4 core modules implemented**

  Ingestion, Detection, Reporting, Orchestrator — tested end-to-end against demo_app fixture

- **LLM provider: Groq API**

  OpenAI-compatible, free-tier open-weight model — chosen for genuine no-credit-card access

- **Strict JSON schema mode**

  reasoning/engine.py enforces structured output from the LLM for reliable parsing

- **Detection rule classes**

  SQLI, SECRET, XSS, AUTH, DESER/SSRF defined in rules.yaml, tree-sitter pinned to 0.21.3

TECH STACK

Analysis

Python AST, tree-sitter

LLM

Groq API (OpenAI-compatible)

Reporting

Jinja2 templating

Testing

pytest + demo_app fixtures

Planned: Agentic Extension to M3

Today's pipeline is deterministic — fixed order, one pass. This is where we introduce a real control loop.

THE LOOP

1 **Propose patch**

2 **Apply to a copy**

3 **Re-run detection**

4 **Decide**

Component Diagram

Stateless, bounded linear pipeline — no database layer by design

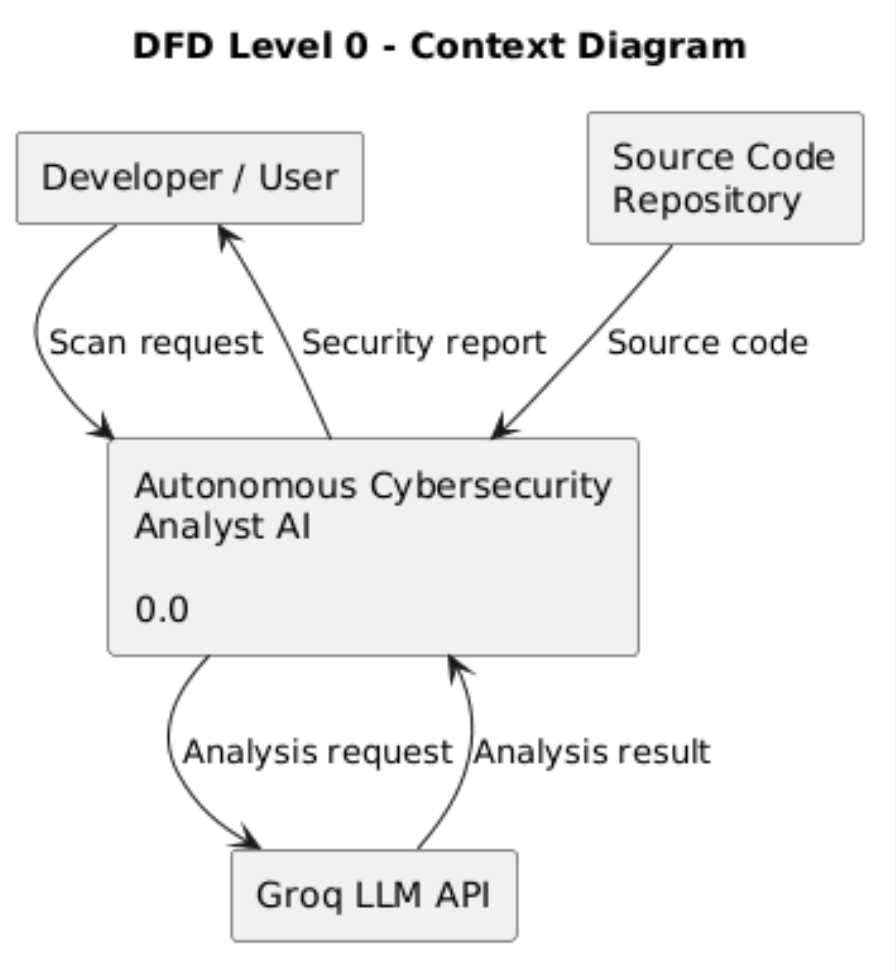
Use Case Diagram

Class Diagram

Activity Diagram

Sequence Diagram

Data Flow Diagram - 0



Data Flow Diagram - 1

References

Sources for the literature survey and tooling comparisons

[1] **Bandit — Python AST-based security linter.** *PyCQA. Bandit Documentation. https://bandit.readthedocs.io*

[2] **Semgrep — static analysis / pattern-matching engine.** *Semgrep Inc. Semgrep Documentation. https://semgrep.dev/docs*

[3] **SonarQube — static analysis and code quality platform.** *SonarSource. SonarQube Documentation. https://docs.sonarsource.com*

[4] **Snyk Code — AI-assisted SAST with Agent Fix (formerly DeepCode AI Fix), context-aware patch suggestions.** *Snyk Ltd. Fix code vulnerabilities automatically. docs.snyk.io, 2026.*

[5] **GitHub Copilot Autofix — LLM-generated fix suggestions for CodeQL code-scanning alerts, part of GitHub Advanced Security.** *GitHub, Inc. About Copilot Autofix for code scanning. docs.github.com, 2026.*

[6] **tree-sitter — incremental parsing library used for AST-based detection.** *tree-sitter contributors. tree-sitter Documentation. https://tree-sitter.github.io*

[7] **Groq API — OpenAI-compatible LLM inference used for the reasoning & patch engine.** *Groq, Inc. GroqCloud API Documentation. https://console.groq.com/docs*

[8] **Jinja2 — templating engine used for report generation.** *Pallets Projects. Jinja Documentation. https://jinja.palletsprojects.com*

Conclusion

Simple pipeline. Clear output. Room to grow.



Thank you